

Chapitre 1 : L'environnement du data scientist, notebooks et outils

Par **Robert DIASSÉ**, Ingénieur Logiciel et Data

> **Cours** : Python pour la science des données · **Fil rouge** : les prix des denrées sur les marchés du Sénégal · **Version** : 2026-10-18

Fiche du chapitre

Objectifs (OS1)	A la fin du chapitre, l'apprenant sait : 1) décrire les étapes d'une analyse de données ; 2) dire à quoi servent Jupyter, NumPy, pandas, Matplotlib et seaborn ; 3) travailler dans un notebook (cellules de code et de texte, ordre d'exécution) ; 4) utiliser Google Colab ; 5) installer un environnement de travail sur son ordinateur (environnement virtuel, `pip`, JupyterLab) ; 6) rédiger un notebook **reproductible**.
Prérequis	Python de L1 : variables, listes, dictionnaires, fonctions
Durée	🟢 Débutant : 6 h · 🔵 Standard : 4 h 30 · 🟣 Expert : 3 h (atelier compris)
Matériel	Un compte Google (pour Colab) ; pour travailler sur son ordinateur : Python 3.12 ou plus récent
Livrable	L'atelier 1 : un premier notebook commenté sur les prix des marchés

Comment lire ce chapitre

- **Noyau** : obligatoire pour tous les niveaux.
- 🟢 **Débutant** : explications pas à pas. A sauter si le public est à l'aise.
- 🔵 **Standard** : le niveau de référence.
- 🟣 **Expert** : approfondissements.

Chaque morceau de code nouveau est décortiqué dans un tableau « Morceau / Ce qu'il veut dire ». Les encadrés 🖍️ **Essayez** vous font provoquer des erreurs volontairement : lisez le message, puis annulez.

Le plan du cours

Ch	Thème	Ce que vous saurez faire sur les prix des marchés
1	**L'environnement de travail**	Ouvrir les données et les décrire

Ch	Thème	Ce que vous saurez faire sur les prix des marchés
2	Fichiers et erreurs	Lire les relevés en Python pur et écarter proprement les lignes abîmées
3	Calcul numérique avec NumPy	Calculer vite sur des milliers de prix
4	Manipulation et nettoyage avec pandas	Sélectionner, filtrer, nettoyer les relevés
5	Agrégation et restructuration	Prix moyen par région, par denrée, par année ; croiser avec la table des marchés
6	Exploration et visualisation	Courbes, comparaisons, conclusions ; mini-projet

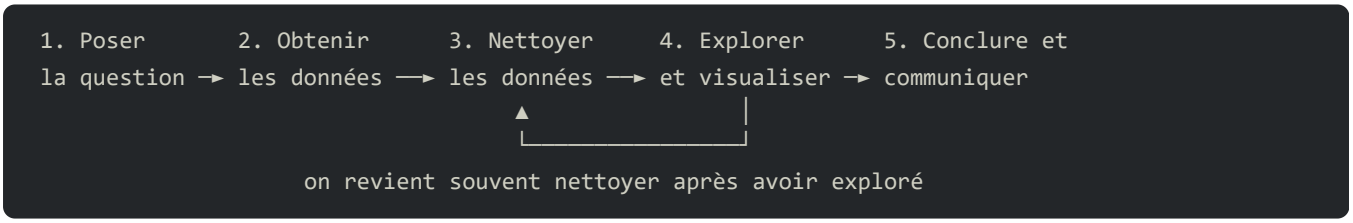
Le **fil rouge** est un vrai jeu de données ouvert : les **prix des denrées relevés sur les marchés du Sénégal** par le Programme alimentaire mondial (PAM), de 2000 à 2026. Il vous est fourni : 52 021 relevés, 67 marchés, 14 régions. Source : World Food Programme, via Humanitarian Data Exchange ; licence CC BY-IGO (réutilisation libre en citant la source).

---

1. La science des données, c'est quoi ?

La **science des données** (en anglais *data science*) consiste à **répondre à des questions à partir de données**. Par exemple : le prix du riz importé a-t-il augmenté depuis 2020 ? Est-il plus cher à Ziguinchor qu'à Dakar ? Le mil est-il moins cher après la récolte ?

Une analyse suit toujours les mêmes étapes :



Étape	Ce qu'on fait	Chapitre du cours
1. Poser la question	Une question précise, à laquelle les données peuvent répondre	Tout le cours
2. Obtenir les données	Trouver, télécharger, charger un fichier	1 et 2
3. Nettoyer	Corriger les types, les valeurs manquantes, les doublons, les erreurs	2 et 4
4. Explorer et visualiser	Calculer des statistiques, regrouper, tracer des graphiques	3, 5 et 6
5. Conclure et communiquer	Écrire ce que l'on a trouvé, avec ses limites	6

> 🟢 **Débutant : où est le travail ?** On imagine le data scientist en train de tracer de beaux graphiques. En réalité, l'essentiel du temps passe dans les étapes 2 et 3 : obtenir des données et les rendre utilisables. C'est pourquoi ce cours y consacre trois chapitres.

2. Les outils

Python est devenu le langage principal de la science des données grâce à ses **bibliothèques** : des ensembles d'outils tout prêts, qu'on installe et qu'on importe.

Outil	A quoi il sert	On l'importe par
Jupyter (notebooks)	Écrire du code, du texte et des graphiques dans un même document	(c'est l'environnement, pas un `import`)
NumPy	Calculer très vite sur de grands tableaux de nombres	`import numpy as np`
pandas	Manipuler des **tableaux de données** : lignes, colonnes, filtres, regroupements	`import pandas as pd`
Matplotlib	Tracer des graphiques	`import matplotlib.pyplot as plt`
seaborn	Des graphiques statistiques plus simples à écrire, construits sur Matplotlib	`import seaborn as sns`

Les abréviations `np`, `pd`, `plt` et `sns` sont des **conventions** suivies par tout le monde : utilisez-les, vos notebooks seront lisibles par n'importe quel data scientist.

3. Le notebook

3.1 Des cellules

Un **notebook** (fichier `.ipynb`) est une suite de **cellules** :

Type de cellule	Contenu	Quand on l'exécute
Code	Du code Python	Le code s'exécute et son résultat s'affiche **sous** la cellule
Texte (Markdown)	Des titres, des paragraphes, des listes	Le texte est mis en forme

On exécute une cellule avec `Maj+Entrée` (elle s'exécute et on passe à la suivante), ou avec le bouton ▶ à sa gauche.

3.2 La mémoire du notebook et l'ordre d'exécution

Toutes les cellules partagent **la même mémoire** : une variable créée dans une cellule existe dans les suivantes. Mais attention, ce qui compte est **l'ordre dans lequel vous avez exécuté les cellules**, pas leur ordre dans la page. Tapez ces trois cellules, une par une :

```
prix_riz = [425, 430, 450, 445]
```

```
moyenne = sum(prix_riz) / len(prix_riz)
print(f"Prix moyen du riz : {moyenne:.1f} F le kilo")
```

```
Prix moyen du riz : 437.5 F le kilo
```

```
prix_riz.append(500)
moyenne
```

```
437.5
```

Morceau	Ce qu'il veut dire
`prix_riz.append(500)`	Ajoute un cinquième prix à la liste
`moyenne` seul, en dernière ligne	Dans un notebook, la **valeur de la dernière ligne** d'une cellule s'affiche automatiquement, sans `print`
La moyenne vaut toujours 437,5	`moyenne` a été calculée **avant** l'ajout de 500 : la variable ne se recalcule pas toute seule. Il faut réexécuter la deuxième cellule

A gauche de chaque cellule de code, un numéro entre crochets (`[1]` , `[2]` ...) indique **dans quel ordre** elle a été exécutée. Une cellule à `[7]` placée au-dessus d'une cellule à `[3]` est le signe d'un notebook exécuté dans le désordre.

> 🧨 **Essayez** : dans une nouvelle cellule, tapez `print(prix_mil)` et exécutez. Le notebook affiche :
>
> ```
> ---------------------------------------------------------------------------
> NameError                                 Traceback (most recent call last)
> Cell In[4], line 1
> ----> 1 print(prix_mil)
>
> NameError: name 'prix_mil' is not defined
> ```
>
> `Cell In[4], line 1` : l'erreur est dans la cellule exécutée en 4e, ligne 1 ; la flèche `---->` montre la ligne. La dernière ligne donne le type et le message : `prix_mil` n'existe pas. Dans un notebook, cette erreur vient souvent d'une cellule **qu'on a oublié d'exécuter**.

🟢 **Débutant : « Redémarrer et tout exécuter »** Pour être sûr qu'un notebook fonctionne, on **redémarre** sa mémoire (tout est effacé) et on exécute **toutes les cellules dans l'ordre**, de haut en bas. Dans Colab : menu **Exécution > Redémarrer la session et tout exécuter**. Dans JupyterLab : **Kernel > Restart Kernel and Run All Cells**. Si une erreur apparaît, votre notebook dépendait d'une cellule supprimée ou exécutée dans le désordre. Faites-le **toujours** avant de rendre un notebook.

4. Travailler dans Google Colab

Google Colab (colab.research.google.com) est un service gratuit de Google : des notebooks qui s'exécutent **sur les serveurs de Google**, sans rien installer. NumPy, pandas, Matplotlib et seaborn y sont déjà installés.

1. Allez sur https://colab.research.google.com et connectez-vous avec un compte Google.
2. **Fichier > Nouveau notebook**. Renommez-le en cliquant sur son nom en haut à gauche (`Untitled0.ipynb`).
3. Le notebook est enregistré automatiquement dans votre **Google Drive**, dossier `Colab Notebooks`.
4. **Ajouter une cellule** : boutons **+ Code** et **+ Texte**.
5. **Importer un fichier de données** : dans le panneau de gauche, icône **Fichiers** (un dossier), puis le bouton **Importer** (une flèche vers le haut), et choisissez le fichier fourni.

Attention : les fichiers importés dans le panneau Fichiers sont **effacés** quand la session s'arrête (après un moment d'inactivité, ou à la fermeture). Le notebook, lui, reste dans votre Drive. A chaque nouvelle session, il faut réimporter le fichier de données, ou le placer dans votre Drive et le lire depuis Drive (encadré 🔵).

🖍️ **Essayez** : sans avoir importé de fichier, exécutez `import pandas as pd` puis `pd.read_csv("prix_marches.csv")`. Vous obtenez un long message qui traverse plusieurs fichiers de pandas, mais la **dernière ligne** suffit : `FileNotFoundError: [Errno 2] No such file or directory: 'prix_marches.csv'` Le fichier n'est pas dans le dossier de travail du notebook. Avec une bibliothèque, les tracebacks sont longs : **lisez toujours la dernière ligne d'abord**.

🔵 **Standard : lire un fichier depuis Google Drive** Pour ne pas réimporter le fichier à chaque session, rangez-le dans votre Drive, puis, dans une cellule :

```
from google.colab import drive
drive.mount("/content/drive")
```

Colab demande l'autorisation d'accéder à votre Drive ; le fichier est ensuite accessible par un chemin comme `"/content/drive/MyDrive/data/wfp_food_prices_sen.csv"`. Ce code ne fonctionne **que** dans Colab.

5. Travailler sur son ordinateur

Colab suffit pour ce cours. Mais un data scientist travaille aussi sur son ordinateur, avec **JupyterLab** (la même chose que Colab, installée chez soi).

5.1 L'environnement virtuel

Chaque projet a besoin de certaines bibliothèques, dans certaines versions. Pour ne pas mélanger les projets, on crée pour chacun un **environnement virtuel** : un dossier qui contient sa propre copie de Python et ses propres bibliothèques.

Dans un terminal, dans le dossier du projet (par exemple `prix-marches`) :

```
python -m venv .venv
```

Puis on **active** l'environnement :

```
.venv\Scripts\activate
```

(sous macOS et Linux : `source .venv/bin/activate`). Le nom `(.venv)` apparaît alors au début de la ligne du terminal.

Commande	Ce qu'elle fait
`python -m venv .venv`	Crée l'environnement virtuel dans un dossier nommé `.venv` (le point au début est une convention pour les dossiers techniques)
`.venv\Scripts\activate`	L'active : à partir de maintenant, `python` et `pip` sont ceux de l'environnement
`deactivate`	Le désactive

5.2 Installer les bibliothèques avec `pip`

Une fois l'environnement activé :

```
pip install jupyterlab numpy pandas matplotlib seaborn
pip freeze > requirements.txt
jupyter lab
```

Commande	Ce qu'elle fait
`pip install ...`	Télécharge et installe les bibliothèques **dans l'environnement actif**
`pip freeze > requirements.txt`	Écrit la liste exacte des bibliothèques installées et de leurs versions dans le fichier `requirements.txt`
`pip install -r requirements.txt`	Sur un autre ordinateur : réinstalle exactement les mêmes versions. C'est ce qui rend un projet **reproductible**

Commande	Ce qu'elle fait
`jupyter lab`	Lance JupyterLab, qui s'ouvre dans le navigateur

> 🖍️ **Essayez** : dans une cellule, tapez `import pandaz` (faute de frappe). Message : `ModuleNotFoundError: No module named 'pandaz'` Même message si la bibliothèque n'est **pas installée** dans l'environnement du notebook : vérifiez l'orthographe, puis l'environnement actif.

> 🔵 **Standard : `uv` , plus rapide** `uv` est un outil récent qui fait le travail de `venv` et de `pip` , beaucoup plus vite : `uv venv` crée l'environnement, `uv pip install pandas` installe. Il s'installe en suivant https://docs.astral.sh/uv/ . Les commandes `venv` et `pip` restent la référence : elles sont fournies avec Python.

> 🟣 **Expert : conda et Anaconda Anaconda** est une distribution qui installe Python, Jupyter et des centaines de bibliothèques en une fois ; son gestionnaire **conda** crée lui aussi des environnements (`conda create -n prix python=3.12`). Très répandu dans le monde scientifique, mais lourd (plusieurs gigaoctets). Pour ce cours, `venv` et `pip` suffisent.

5.3 Vérifier ses versions

Dans une cellule :

```
import sys
print(sys.version)
```

```
import numpy as np
import pandas as pd
import matplotlib
print("NumPy", np.__version__, "| pandas", pd.__version__, "| Matplotlib", matplotlib.__version__)
```

Vous obtenez par exemple `3.12.13 (main, ...)` et `NumPy 2.5.4 | pandas 3.0.6 | Matplotlib 3.11.2` (vos numéros peuvent être différents, en particulier dans Colab).

Morceau	Ce qu'il veut dire
`import sys`	Le module `sys` donne des informations sur Python lui-même
`np.__version__`	Chaque bibliothèque connaît son numéro de version. Le noter en haut d'un notebook permet de comprendre plus tard pourquoi un résultat a changé

6. Écrire du texte dans un notebook : le Markdown

Une cellule de texte s'écrit en **Markdown**, un langage très simple : vous tapez quelques symboles, la mise en forme apparaît quand vous exécutez la cellule.

Vous tapez	Vous obtenez
`# Titre`, `## Sous-titre`, `### Sous-sous-titre`	Des titres de trois niveaux
`**important**`	**important** (en gras)
`*mot*`	*mot* (en italique)
`- élément` sur plusieurs lignes	Une liste à puces
`1. étape`	Une liste numérotée
`` `code` ``	`code` (en police de code)
`[texte](https://...)`	Un lien

Un bon notebook **raconte** l'analyse : un titre, l'objectif, la source des données, puis, entre les cellules de code, ce que l'on fait et ce que l'on observe.

7. Un notebook reproductible

Un notebook est **reproductible** si n'importe qui (vous dans six mois, l'enseignant, un collègue) peut le rouvrir, l'exécuter et obtenir **les mêmes résultats**. Les règles :

1. **Un en-tête** : titre, auteur, date, objectif, **source et licence des données**.
2. **Tous les `import` dans la première cellule de code**.
3. **Les versions** de Python et des bibliothèques, affichées en haut.
4. **Un nom de fichier dans une variable** (`FICHIER = "..."`), écrit une seule fois, avec un chemin **relatif** (pas `C:\Users\Awa\Bureau\...`, qui n'existe que sur votre ordinateur).
5. **Les cellules dans l'ordre** : chaque cellule n'utilise que ce qui est créé au-dessus d'elle.
6. **Des explications** en Markdown entre les cellules, et des **conclusions** écrites, pas seulement des chiffres.
7. **« Redémarrer et tout exécuter »** sans erreur avant de rendre.
8. **Pas de données personnelles** (noms, téléphones) dans un notebook partagé.

> 🟣 **Expert : mesurer le temps d'une cellule** Les notebooks ont des commandes spéciales, qui commencent par `%`. `%timeit sum(range(1000))` exécute l'instruction des milliers de fois et affiche son temps moyen, par exemple `25.9 µs ± 1.28 µs per loop (mean ± std. dev. of 7 runs, 10,000 loops each)`. Vous vous en servirez au chapitre 3 pour comparer une boucle Python et NumPy. Dans Colab, `!pip install nom` installe une bibliothèque manquante (le `!` exécute une commande du terminal).

8. Premier contact avec les données du fil rouge

Importez le fichier fourni `wfp_food_prices_sen.csv` (dans Colab : panneau Fichiers), puis :

```
import pandas as pd

releves = pd.read_csv("wfp_food_prices_sen.csv")
releves.shape
```

```
(52021, 16)
```

```
releves.head(3)
```

```
         date admin1 admin2  market  market_id  latitude  longitude  \
0  2000-01-15  Dakar  Dakar  Tilene        447     14.68     -17.46
1  2000-01-15  Dakar  Dakar  Tilene        447     14.68     -17.46
2  2000-01-15  Dakar  Dakar  Tilene        447     14.68     -17.46

             category         commodity  commodity_id unit priceflag  \
0  cereals and tubers  Maize (imported)           158   KG    actual
1  cereals and tubers           Millet            73   KG    actual
2  cereals and tubers   Rice (imported)            64   KG    actual

  pricetype currency   price  usdprice
0    Retail      XOF  149.00      0.35
1    Retail      XOF  124.75      0.30
2    Retail      XOF  227.50      0.54
```

(Dans un notebook, le tableau s'affiche avec une mise en forme plus agréable.)

Morceau	Ce qu'il veut dire
`pd.read_csv("...")`	Lit un fichier CSV et le range dans un **DataFrame** : un tableau avec des lignes et des colonnes nommées. Vous l'étudierez en détail au chapitre 4
`releves.shape`	Sa taille : `(nombre de lignes, nombre de colonnes)` : 52 021 relevés, 16 colonnes
`releves.head(3)`	Les 3 premières lignes (5 par défaut). Le `\` en fin de ligne indique que le tableau, trop large, continue en dessous
`0`, `1`, `2` à gauche	L'**index** : le numéro de chaque ligne

Les colonnes principales :

Colonne	Contenu
`date`	Le mois du relevé (toujours le 15 du mois)
`admin1`, `admin2`	La région et le département
`market`, `latitude`, `longitude`	Le marché et sa position
`category`, `commodity`	La catégorie et la denrée (en anglais : *Millet* = mil, *Rice (imported)* = riz importé, *Maize* = maïs, *Sorghum* = sorgho, *Beans (niebe)* = niébé, *Groundnuts* = arachide)

Colonne	Contenu
`unit`	L'unité du prix (le kilo, `KG`, presque toujours)
`pricetype`, `currency`, `price`	Le type de prix (détail, `Retail`), la monnaie (`XOF`, le franc CFA) et le prix
`usdprice`	Le même prix converti en dollars américains

Ce n'est qu'un aperçu : l'atelier vous fait poser vos premières questions à ces données.

Exercices

Dans un nouveau notebook Colab nommé `exercices_ch1.ipynb`.

- ★ **Exercice 1 : le Markdown.** Écrivez une cellule de texte qui contient : un titre « Mes premières notes », un sous-titre, une liste à puces des cinq outils de la section 2 avec, en gras, le nom de chacun, et un lien vers https://pandas.pydata.org . (Section 6.)
- ★ **Exercice 2 : l'ordre d'exécution.** Créez trois cellules : `a = 10` ; `b = a * 2` ; `print(a, b)`. Exécutez-les dans l'ordre 1, 3, 2, puis 3 : notez ce qui s'affiche à chaque fois et les numéros entre crochets. Expliquez en Markdown. Puis faites « Redémarrer et tout exécuter ». (Section 3.2.)
- ★★ **Exercice 3 : un petit calcul de prix.** Dans une cellule, créez un dictionnaire des prix du kilo de cinq denrées que vous achetez (riz, huile, sucre, oignon, lait en poudre, avec des prix de votre choix). Dans les cellules suivantes, calculez le prix moyen, la denrée la plus chère et la plus chère que la moyenne, avec du **Python pur** (pas de pandas). Racontez chaque étape en Markdown. (Sections 3 et 6.)
- ★★ **Exercice 4 : sur votre ordinateur.** Créez un dossier `prix-marches`, un environnement virtuel, installez `jupyterlab` et `pandas`, produisez un `requirements.txt`, lancez JupyterLab et refaites l'exercice 3. Ouvrez `requirements.txt` : combien de bibliothèques contient-il, alors que vous n'en avez demandé que deux ? Pourquoi ? (Section 5.)
- ★★★ **Exercice 5 : la critique d'un notebook.** L'enseignant vous fournit un notebook « mal fait ». Listez tout ce qui le rend **non reproductible** (avec la liste de la section 7), corrigez-le, et vérifiez qu'il passe « Redémarrer et tout exécuter ». (Section 7.)

Atelier 1 : un premier notebook reproductible sur les prix des marchés

Le travail principal du chapitre : vous construisez, étape par étape, un premier notebook propre et reproductible qui décrit le jeu de données du fil rouge. L'énoncé de l'atelier vous est fourni séparément.

A retenir

- Une analyse de données suit cinq étapes : question, données, nettoyage, exploration et visualisation, conclusions. Le nettoyage prend souvent le plus de temps.
- Les outils : Jupyter (notebooks), NumPy (`np`), pandas (`pd`), Matplotlib (`plt`), seaborn (`sns`).

- Un notebook est une suite de cellules de code et de texte (Markdown), qui partagent la même mémoire ; ce qui compte est **l'ordre d'exécution** (les numéros `[n]`).
- Colab : rien à installer, mais les fichiers importés disparaissent à la fin de la session.
- Sur son ordinateur : un **environnement virtuel** par projet (`python -m venv .venv`), `pip install`, `pip freeze > requirements.txt`.
- Un notebook reproductible : en-tête et source, imports et versions en haut, chemins relatifs, cellules dans l'ordre, explications, et « Redémarrer et tout exécuter » sans erreur.
- Avec une bibliothèque, un traceback est long : lisez la **dernière ligne** d'abord.

Erreurs fréquentes

Message ou symptôme	Ce que ça veut dire	Solution
`NameError: name '...' is not defined`	Une cellule qui crée la variable n'a pas été exécutée (ou a été supprimée)	Exécuter les cellules au-dessus ; « Redémarrer et tout exécuter »
Un résultat ne change pas après une modification	La cellule de calcul n'a pas été réexécutée	Réexécuter les cellules qui dépendent de la modification
`ModuleNotFoundError: No module named '...'`	Faute de frappe, ou bibliothèque non installée dans l'environnement du notebook	Corriger le nom ; `pip install` dans l'environnement actif (ou `!pip install` dans Colab)
`FileNotFoundError: [Errno 2] No such file or directory: '...'`	Le fichier n'est pas dans le dossier de travail (dans Colab : pas importé, ou effacé en fin de session)	Réimporter le fichier ; vérifier le nom et le chemin
`(.venv)` n'apparaît pas dans le terminal	L'environnement n'est pas activé	`.venv\Scripts\activate`
Le notebook marche chez vous, pas chez l'enseignant	Chemin absolu, cellule exécutée dans le désordre, bibliothèque manquante	Les règles de la section 7

Pour aller plus loin

- Bienvenue dans Colab (guide officiel, en français) : https://colab.research.google.com/notebooks/intro.ipynb
- La documentation de JupyterLab : https://jupyterlab.readthedocs.io
- Les environnements virtuels, dans le tutoriel officiel de Python (en français) : https://docs.python.org/fr/3/tutorial/venv.html
- Le guide de démarrage de pandas : https://pandas.pydata.org/docs/getting_started/index.html
- La source du fil rouge : https://data.humdata.org/dataset/wfp-food-prices-for-senegal